

markpublish Benutzerhandbuch

Moderne PDF- und HTML-Dokumentenerstellung aus Markdown

Dieses Handbuch bietet eine praxisorientierte Anleitung und vollständige Referenz zur Konfiguration, Template-Anpassung und Dokumentengenerierung mit markpublish.

Version	1.0.0
Datum	02.09.2026
Autor	Frank Winter
Copyright	© 2026 Frank Winter

Inhaltsverzeichnis

Einführung & Architektur	4
Motivation & Zielsetzung	4
Die Verarbeitungs-Pipeline	4

KAPITEL 1

Einführung & Architektur

Überblick über Zielsetzung, Kernkonzepte und die modulare Verarbeitungs-Pipeline.

Einführung & Architektur

Willkommen beim **markpublish Benutzerhandbuch**. **markpublish** ist ein modernes, quelloffenes Publishing-Werkzeug, das aus einfachen Markdown-Dateien professionell gesetzte **PDF-Publikationen** und **HTML-Vorschauen** erzeugt.

Motivation & Zielsetzung

Viele Dokumentationswerkzeuge erfordern entweder komplexe LaTeX-Setups oder beschränken sich auf reine HTML-Webseiten. **markpublish** verbindet die Einfachheit von Markdown mit der typografischen Präzision von **CSS Paged Media** über die [WeasyPrint](#)-Engine.

Kernvorteile auf einen Blick:

- **Modulare Kapitel:** Jedes Kapitel wird als separate Markdown-Datei gepflegt.
- **Zweistufige Gliederung:** Übergeordnete Abschnitte (Parts) über den Kapiteln; die Tiefe innerhalb eines Kapitels kommt aus dessen Überschriften.
- **Deklarative Steuerung:** Ein zentrales Manifest (`markpublish.yaml`) steuert Inhalt, Metadaten und Layout.
- **W3C CSS Paged Media:** Exakte Kontrolle über `@page` -Ränder, mehrzeilige Kopf- und Fußzeilen, Seitenzahlen und Trennseiten.
- **Erweiterbare Pipeline:** Saubere Trennung zwischen Parser, Renderer und Template-Ebenen.

Die Verarbeitungs-Pipeline

Die Architektur von **markpublish** folgt einem mehrstufigen Prozess:

1. **Konfigurations-Lader** (`config.loader`): Liest die YAML-Struktur ein, validiert Datentypen und löst dynamische Variablen (z. B. `date: auto`) auf.
2. **Template-Resolver** (`templates.resolver`): Sucht nach der 3-stufigen Priorität (User > Common/Workspace > Package) nach dem passenden Theme für das Zielformat.
3. **Markdown- & TOC-Engine** (`markdown.engine`): Parst Markdown mit erweiterten Erweiterungen (Callouts, Tabellen, Pygments-Syntax-Highlighting), vergibt konsistente Überschriftennummern (`1.1` , `1.2`) und baut Inhaltsverzeichnisse auf.
4. **Renderer** (`renderers.pdf` & `renderers.html`): Übergibt die gerenderten HTML- und CSS-Fragmente an WeasyPrint für den PDF-Export oder speichert eigenständiges, responsives HTML.